

PCG Arena

A Web Platform for Blind A/B Testing of
Procedural Content Generators

Antoni Czapski

Artificial Intelligence ♡ Games:
Procedural Content Generation

February 2026

Outline

- 1 Motivation & Problem
- 2 Platform Overview
- 3 Generators Under Test
- 4 Experiments & Results
- 5 Judge Function & MarioDPO
- 6 Conclusions

Outline

- 1 Motivation & Problem
- 2 Platform Overview
- 3 Generators Under Test
- 4 Experiments & Results
- 5 Judge Function & MarioDPO
- 6 Conclusions

The PCG Evaluation Problem

Challenge: How do we know if a PCG algorithm produces “good” levels?

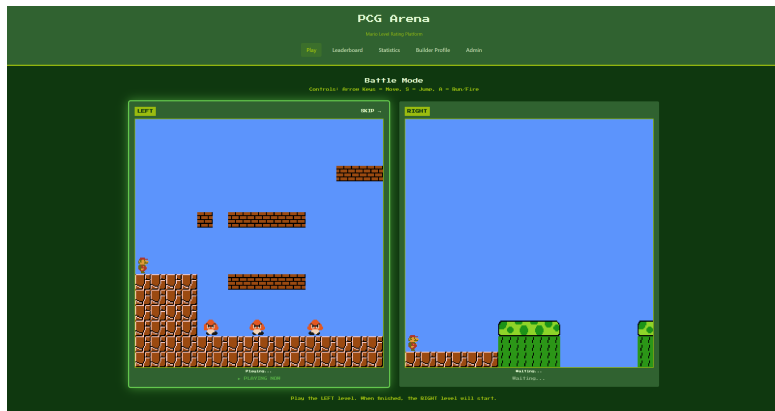
- Automated metrics miss **player experience**
- Lab studies have **limited participants**
- Generator reputation creates **bias**

Solution: Web-based blind A/B testing at scale

Research Questions

- 1 What makes levels “good”?
- 2 Are player preferences consistent?
- 3 Can we build a reward model?

PCG Arena: Core Concept



- Play two levels → Vote for favorite → Glicko-2 ranking
- **15 generators, 748 levels, 571 votes** collected

Outline

- 1 Motivation & Problem
- 2 Platform Overview**
- 3 Generators Under Test
- 4 Experiments & Results
- 5 Judge Function & MarioDPO
- 6 Conclusions

System Architecture

Frontend

- React + TypeScript
- Mario engine ported from Java
- Browser-based gameplay

Backend

- FastAPI (Python)
- Glicko-2 rating system
- AGIS matchmaking

Infrastructure

- Docker containers
- Google Cloud Platform
- SQLite database

Live at:

<https://pcg-arena.com>

Glicko-2 Rating System

Why Glicko-2?

- Provides **uncertainty estimates** (RD)
- Handles **inactive** generators
- Maps pairwise comparisons to rankings

Win probability:

$$P(A \text{ wins}) = \frac{1}{1 + 10^{(R_B - R_A)/400}}$$

Parameter	Value
Initial Rating	1000
Initial RD	350
System τ	0.5

Table: Glicko-2 settings

Outline

- 1 Motivation & Problem
- 2 Platform Overview
- 3 Generators Under Test**
- 4 Experiments & Results
- 5 Judge Function & MarioDPO
- 6 Conclusions

15 PCG Algorithms Compared

Neural/ML-Based

- MarioGPT (GPT-2)
- MarioGAN (DCGAN)
- MarioDiffusion
- MarioDPO (**ours**)

Search-Based

- Genetic algorithm
- Hopper (agent simulation)

Grammar/Pattern-Based

- Notch variants (3)
- Pattern-based (3)
- ORE (occupancy)

Baseline

- **Original SMB** (15 levels)

100 levels per generator (except Original: 15)

Outline

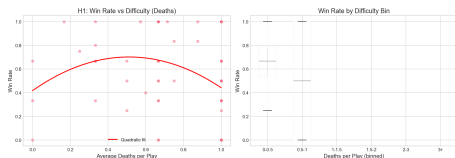
- 1 Motivation & Problem
- 2 Platform Overview
- 3 Generators Under Test
- 4 Experiments & Results**
- 5 Judge Function & MarioDPO
- 6 Conclusions

RQ1: What Makes a Good Level?

H1: Flow Channel Hypothesis

Hypothesis: Moderate difficulty (15–40% death rate) maximizes enjoyment.

Result: **Not Supported** – Linear relationship: easier = better
($r = -0.227$, $p = 0.046$)



Conclusion

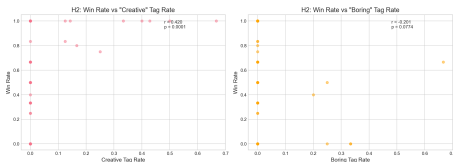
Players prefer levels they can **complete**. No inverted-U pattern found.

RQ1: Structural Variety

H2: Creativity Hypothesis

Hypothesis: Higher terrain variety and enemy diversity preferred.

Result: **Partially Supported** – “Creative” tag $\leftrightarrow$ wins ($r = 0.42$, $p = 0.0001$)



Tag	r	p
Creative	0.416	0.003
Boring	-0.246	0.092

Conclusion

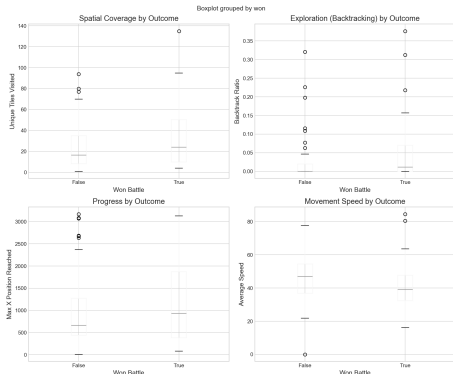
Levels tagged “creative” win significantly more often.

RQ1: Path Freedom & Exploration

H3: Agency Hypothesis

Hypothesis: Levels allowing more exploration are preferred.

Result: **Partial Support** – Backtrack ratio predicts wins ($p = 0.019$)



Metric	Won	Lost
Unique Tiles	68.7	42.1
Backtrack %	13%	21%
Max X	1533	1095

Conclusion

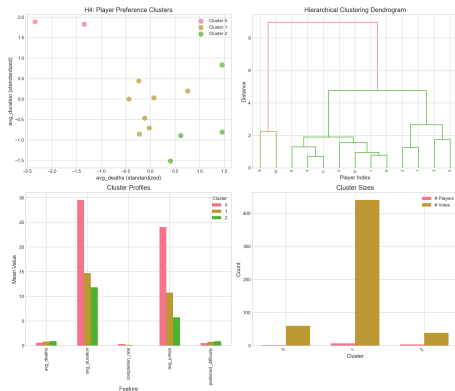
Winners explore **63% more tiles**.
Non-linear paths preferred.

RQ2: Player Consistency

H4: Player Clusters Hypothesis

Hypothesis: Distinct player preference groups exist.

Result: **Supported** – 3 clusters identified via K-means



Player Types:

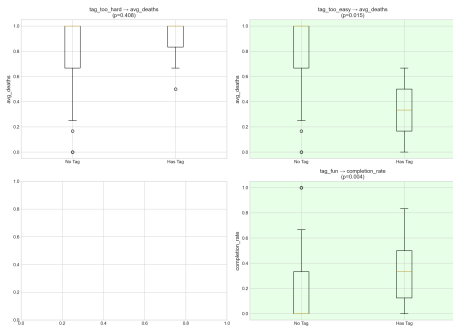
- 1 **Explorers** (n=2)
Long sessions, high completion
- 2 **Mainstream** (n=7)
Most votes, balanced play
- 3 **Strugglers** (n=4)
Low completion, high deaths

RQ3: Tag Validation

H6: Tags Match Telemetry

Hypothesis: Subjective tags correlate with objective metrics.

Result: **Supported** – 5/6 tags match expected metric direction



Tag Semantics:

- fun $\approx$ playable + beatable
- creative $\approx$ engaging
- too_easy $\approx$ high completion
- too_hard $\approx$ high death rate

Conclusion

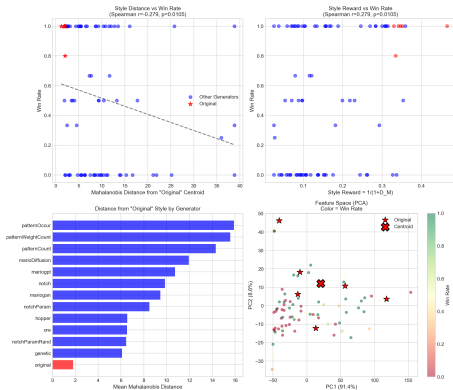
Tags can be **trusted** as quality signals for ML.

RQ1: Style Matching

Exp D: Mahalanobis Distance

Hypothesis: Levels closer to “Original” style win more.

Result: **Supported** – $r = -0.279$, $p = 0.011$



Conclusion

The “Nintendo Factor”: original SMB levels define the **quality target**.

Style reward:

$$r_{style} = \frac{1}{1 + D_M}$$

where D_M = Mahalanobis distance

Outline

- 1 Motivation & Problem
- 2 Platform Overview
- 3 Generators Under Test
- 4 Experiments & Results
- 5 Judge Function & MarioDPO**
- 6 Conclusions

Building an Automated Judge Function

Goal: Create reward model for RLHF/DPO training

Stage 1 – Static Features:

$$J_{static} = \frac{w_{style}}{1 + D_M} - w_{gap} \cdot \text{Gap}$$

Weight	Value	Source
w_{style}	0.32	Exp D
w_{vert}	0.26	Exp A
w_{gap}	0.50	Exp B

Stage 2 – Dynamic Features:

$$J_{final} = J_{static} + w_{vert} \cdot \sigma_y$$

Key Result

Judge function correlates with human preference: **Spearman** $r = 0.736$,
 $p = 0.003$

MarioDPO: Aligning Generators with Human Preferences

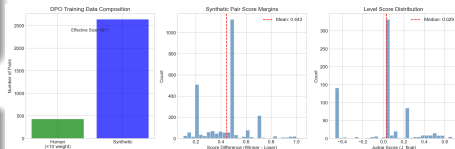
Approach: Direct Preference Optimization (DPO) fine-tuning

Training Data

- 473 human pairs ($\times 10$ weight)
- 2,606 synthetic pairs (Judge)
- **7,336 effective pairs**

Architecture

- Base: GPT-2 small
- Tokenization: Character-level
- DPO $\beta = 0.1$



MarioDPO Results: Beating Other PCG Methods

PCG Arena
Mario-Led Rating Platform

Play **Leaderboard** Statistics Builder Profile Admin

Generator Leaderboard
Rankings based on player votes using the rating system

Full Leaderboard

Rank	Generator	Rating	Games	W	L	T
1	Original Super Mario Bros (SMB) Levels	1491	83	70	6	7
2	MarioDPO (Direct Preference Optimization)	1340	20	15	3	2
3	Occupancy-Regulated Extension (ORE)	1221	91	55	25	11
4	Notch (Infinite Mario Bros) Generator	1155	77	44	23	10
5	Hopper Level Generator	1092	50	42	30	5
6	Grammatical Evolution (GE) Generator	1065	75	35	27	16
7	MarioDPO (Text2Level via GPT-4)	1043	38	49	32	7
8	Reinforce (DCGM + ON-ES Latent-Space Generator)	1020	91	39	41	11
9	Pattern-based Generator (Weighted Pattern Count Fitness)	936	63	22	42	12
10	MarioDiffusion (Text-to-Level Diffusion)	936	54	26	40	15
11	Parameterized Notch (Randomized Params) Generator	909	55	27	43	16
12	Parameterized Notch Generator	855	52	16	49	17
13	Pattern-based Generator (Pattern Sequence Fitness)	841	79	19	51	9
14	Pattern-based Generator (Pattern Count Fitness)	777	77	11	50	16

Updated 2/20/25 10:00PM

Key Achievement

MarioDPO achieves highest Elo among all PCG generators, second only to hand-crafted Original SMB levels!

Generator Rankings Summary

Rank	Generator	Win Rate
1	Original (SMB)	89.6%
2	MarioDPO (ours)	83.3%
3	ORE	69.3%
4	Notch	67.4%
5	MarioGPT	63.8%
12	patternOccur	26.1%
13	notchParam	23.9%
14	patternCount	20.4%

Key Insights

- Original dominates (89%)
- **MarioDPO** beats other ML methods
- Pattern generators fail
- Style distance predicts quality

Outline

- 1 Motivation & Problem
- 2 Platform Overview
- 3 Generators Under Test
- 4 Experiments & Results
- 5 Judge Function & MarioDPO
- 6 Conclusions**

Key Findings

① **Playability > Challenge**

Players prefer levels they can complete (no flow channel)

② **Exploration Matters**

Non-linear, explorable levels win 63% more tiles visited

③ **Style Matching Works**

Closer to Original SMB $\rightarrow$ higher win rate ($r = 0.28$)

④ **Tags are Valid Signals**

Subjective tags correlate with objective metrics (5/6)

⑤ **DPO Alignment Succeeds**

MarioDPO achieves **highest Elo** among PCG generators

Contributions

Platform

- Open-source web platform for PCG evaluation
- Mario engine ported Java → TypeScript
- Glicko-2 + AGIS matchmaking

Research

- Empirical comparison of 15 generators
- Validated judge function ($r = 0.74$)
- MarioDPO: SOTA among PCG methods (83.3%)

Data

- 748 levels, 571 human votes
- Player trajectories and death heatmaps
- Open dataset for future research

Short-term

- Collect more votes
- Add more generators
- Improve matchmaking

Long-term

- Transfer to other games
- Online DPO training
- Multi-objective PCG

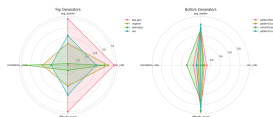
<https://pcg-arena.com>

Try it yourself!

Questions?

Platform: <https://pcg-arena.com>

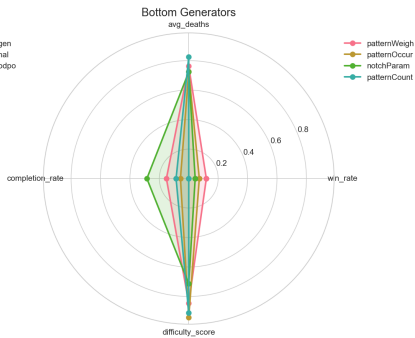
Code: <https://github.com/antoniczapski/pcg-arena>



Backup: Generator Fingerprints

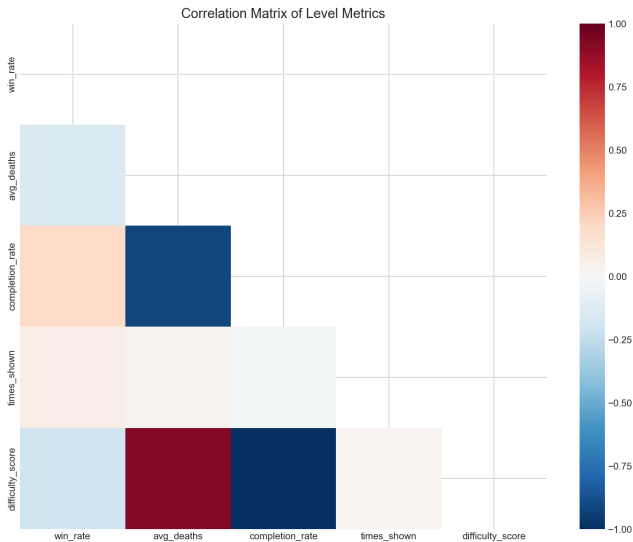


test-gen
original
mariodpo
ore



patternWeigh
patternOccur
notchParam
patternCount

Backup: Correlation Matrix



Backup: Feature Importance

